Beyond Static Segmentation: LSTM-Enhanced Architectures for Temporal Consistency in Root Analysis

Tudor Pitulice[1], Alican Noyan, PhD.[1], Shival Indermun, PhD.[1]

[1]Data Science and Artificial Intelligence, Breda University of Applied Sciences, Mgr. Hopmansstraat 2, The Netherlands.

Contributing authors: tudor.pitulice@gmail.com; @gmail.com; @gmail.com;

Abstract

Current deep learning approaches for root system segmentation rely on static convolutional architectures that process each frame independently, inheriting the characteristic limitations of their backbone: noise sensitivity in U-Net, under-segmentation in ResNet, and diffuse attention in Vision Transformers. This study investigates whether integrating ConvLSTM2D layers into these architectures can mitigate such limitations while introducing temporal consistency across time-series imagery of Arabidopsis thaliana root systems. Using a controlled replacement strategy that isolates temporal recurrence as the sole variable, we evaluate LSTM-enhanced variants of U-Net, ResNet, and ViT on both standard segmentation metrics and a newly proposed Temporal Consistency Score (TCS), a cascade-based penalty metric quantifying frame-to-frame prediction coherence against ground-truth root growth. Results reveal that the benefit of temporal integration is architecture-dependent: LSTM layers improved performance and consistency in U-Net and ViT but degraded temporal coherence in ResNet, where residual connections bypassed the recurrent layer during training. RISE-based explainable AI analysis corroborates these findings, showing how temporal context reshapes spatial attention differently depending on the host architecture's information flow. These results caution against treating temporal layers as a universal enhancement and highlight the need for architectural compatibility analysis in phenotyping pipeline design.

Keywords: root segmentation, temporal consistency, ConvLSTM2D, U-Net, ResNet, Vision Transformer, Arabidopsis thaliana, explainable AI, plant phenotyping

1 Introduction

Root system architecture (RSA) plays a critical role in plant adaptation to environmental stresses, nutrient acquisition, and overall crop productivity [1–3]. Understanding and manipulating RSA is essential for developing climate-resilient crops capable of withstanding abiotic stresses such as drought and nutrient deficiency while maintaining high yields [1, 2]. High-throughput phenotyping platforms have revolutionized root analysis by enabling automated, non-destructive measurement of root traits through advanced imaging technologies combined with deep learning approaches [4, 5]. These systems generate time-series datasets that capture root development across multiple days or weeks, providing unprecedented insights into dynamic growth processes and stress responses [4].

Despite the availability of temporal data, current segmentation architectures process each frame independently, discarding the sequential information that phenotyping pipelines inherently produce. Without temporal context, segmented roots can disappear or fragment between timepoints, destabilising length measurements and lateral root counts across the sequence [4, 5]. This temporal inconsistency undermines the longitudinal trait quantification that high-throughput phenotyping is designed to deliver [4, 6].

This study investigates whether integrating Long Short-Term Memory (LSTM) layers into established segmentation architectures can address this problem structurally. We present a systematic comparative analysis of LSTM-enhanced variants of U-Net, ResNet, and Vision Transformer (ViT) for temporally consistent root segmentation in *Arabidopsis thaliana*, using a controlled replacement strategy that isolates temporal recurrence as the sole experimental variable. Models are evaluated on traditional frame-wise metrics (F1-score, mIoU, precision and recall) alongside a newly proposed Temporal Consistency Score (TCS) that quantifies prediction stability across the full time series. Furthermore, we employ RISE-based explainable AI (XAI) analysis [7] to reveal how temporal recurrence reshapes each architecture's spatial attention, providing mechanistic insight into when and why LSTM integration succeeds or fails.

2 Literature Review

2.1 Deep learning for root system segmentation

Early computer vision approaches for root system architecture (RSA) analysis relied on manual or semi-automatic tools that were time-consuming, user-dependent, and difficult to scale to high-throughput settings. RootNav 2.0 addressed these limitations with an encoder–decoder convolutional neural network (CNN) that performs multi-class pixel-wise segmentation, separating background, first-order roots, second-order roots, and keypoint heatmaps for seeds and root tips. The segmentation output is then converted into a full RSA representation through graph-based reconstruction and exported in RSML format [8].This multi-task design demonstrated that deep CNNs can robustly extract complex root geometries across species and imaging modalities while offering substantial speed improvements over semi-automatic workflows [8].

Subsequent work has explored diverse encoder–decoder designs for in situ root segmentation. Shen et al. [9] employed an improved DeepLabv3+ with atrous spatial pyramid pooling (ASPP) to segment rhizotron images, outperforming U-Net and traditional pipelines by better capturing multi-scale root context in soil backgrounds. U-Net variants have been widely refined for root segmentation across species and substrates, leveraging skip connections and multi-scale fusion to preserve thin root structures [6, 10]. Recent reviews synthesise these advances, highlighting strong CNN performance for static frame segmentation alongside challenges in generalisation and temporal consistency [5].

Despite these successes, most RSA segmentation methods evaluate on individual frames using frame-wise metrics (IoU, Dice/F1), with temporal consistency achieved mainly through post-processing rather than explicit temporal modeling within the segmentation network [4, 11–13].

2.2 Architecture-specific limitations in segmentation

Each major segmentation backbone exhibits characteristic weaknesses that impact root analysis. Standard U-Net architectures propagate shallow, high-resolution noise through skip connections, leading to oversegmentation and false positives when backgrounds contain fine textures or artefacts [11]. False positives have been reported as a persistent challenge in plant root segmentation, with background noise at shallow layers causing spurious foreground predictions that require auxiliary classification or attention modules to suppress [11, 14].

In contrast, ResNet-based encoder backbones suffer from progressive downsampling and pooling that attenuate information about small or thin structures, resulting in missed objects and false negatives [15, 16]. Studies on thin-structure plant segmentation consistently report that conventional encoder–decoder networks undersegment fine targets unless augmented with specialised attention mechanisms or high-resolution paths [15, 16].

ViTs and transformer-based segmenters excel at global context but often produce coarse boundaries and spatially diffuse attention due to low-resolution feature grids and lack of local inductive bias [17, 18]. ViT-based plant analysis approaches have been shown to require larger training datasets than CNN counterparts and often produce relatively coarse recognition due to fixed patch tokenisation and limited local feature extraction, motivating hybrid or spatially-modulated attention strategies to sharpen focus on target structures [17, 18].

2.3 Temporal modeling in plant phenotyping

Temporal deep learning has gained traction for dynamic plant phenotyping. ChronoRoot combines high-throughput temporal imaging with CNN segmentation and post-processing filters to extract novel dynamic RSA descriptors, but relies on frame-independent predictions refined retrospectively [4]. Yasrab et al. proposed CNN–LSTM frameworks that feed convolutional root features to recurrent layers for genotypic classification, demonstrating temporal modeling improves discriminative power [19]. Tian

et al. developed U+LSTM-F, where U-Net features drive LSTM-based rice seedling growth prediction from time-series data [20].

LSTM applications extend to disease progression and temporal classification, reinforcing sequence modeling benefits for agricultural imaging [21]. However, these works target classification or growth prediction from summary features rather than dense, temporally consistent pixel-wise segmentation.

Post-processing approaches operate on already-degraded predictions, and their corrections cannot recover information that the segmentation network never extracted. By contrast, integrating temporal recurrence directly within the architecture allows the network to leverage cross-frame context during feature learning itself, in principle producing more stable predictions before any post-processing is applied.

2.4 Hybrid architectures and explainable AI

Hybrid transformer–recurrent models have been developed in other fields such as medicine. Hossain et al. developed ViT–LSTM for brain stroke detection in CT images, where ViT extracts global slice features and LSTM models temporal or augmented-view sequences, outperforming standalone CNNs and pure ViTs [22]. They integrate XAI (attention maps, SHAP, LIME) to validate clinical interpretability, demonstrating hybrid architectures enhance both performance and explainability [22].

In RSA analysis, transformers capture long-range root dependencies [5, 23], but remain frame-wise without temporal recurrence. Plant phenotyping XAI applications (saliency, Grad-CAM) rarely dissect how temporal components reshape backbone failure modes under noisy phenotyping conditions [5].

The RISE framework provides model-agnostic pixel-wise importance maps by correlating outputs with randomized input masks, ideal for comparing static versus temporally enhanced segmentation models [7].

2.5 Research gaps addressed by this work

The existing literature highlights a variety of static models that inherit the architectural weaknesses of their backbone design. The prevailing approach to achieving optimal performance is to compensate for these weaknesses by scaling training data. This study takes a different approach: rather than relying on larger datasets, it investigates whether integrating temporal recurrence directly into the architecture can address these limitations structurally, leveraging the sequential nature of the data that phenotyping pipelines already produce.

No systematic study has examined the effects of LSTM layers on classic CNN architectures (U-Net, ResNet encoders, ViT) specifically for temporally consistent root segmentation. While LSTM integration shows promise in classification and growth prediction [19, 20], its impact on addressing architecture-specific failure modes: U-Net false positives from noise, ResNet false negatives on fine roots, and ViT diffuse attention, remains unexplored [11, 16, 17]. This work fills these gaps by: (i) systematic comparative evaluation of LSTM-enhanced variants of U-Net, ResNet, and ViT architectures for *Arabidopsis thaliana* RSA time-series segmentation under controlled experimental conditions, (ii) quantitative assessment of temporal consistency alongside

traditional frame-wise performance metrics to evaluate segmentation stability across time-series data, (iii) application of RISE-based explainable AI to reveal how temporal recurrence mitigates each backbone's distinct weaknesses, enabling temporally stable root phenotyping.

3 Methodology

In the following section, each architecture scenario is presented with their specific design choices as well as the composition of the data used for conducting the study, and the formula for computing the custom Temporal Consistency Score (TCS).

All experiments were conducted on the Breda University of Applied Sciences server, equipped with a NVIDIA RTX 6000 Ada Generation GPU (48GB of VRAM).

3.1 Data description

The data for this study consists of time-series root images acquired at the National Plant Eco-phenotyping Center (NPEC), capturing *Arabidopsis thaliana* grown on Agar plates under controlled conditions. In total, 600 8-bit greyscale images of original resolution 2643×1891 pixels were collected from two independent experiments, each comprising 20 plates imaged over 15 consecutive days. Each image contains a single Agar plate with five plants, providing densely populated root systems for segmentation; from this dataset, 105 images spanning seven complete 15-day imaging sequences were manually annotated, providing dense temporal coverage suitable for sequence-level training and evaluation. In figure 1 an image from the training set can be found.



Fig. 1: Example image from the training set

The dataset was split into training (75 images from 5 plates), validation (15 images from 1 plate), and test (15 images from 1 plate) subsets to ensure robust model evaluation across independent samples. This split provided sufficient training data while maintaining complete 15-day cycles in validation and test sets for temporally realistic evaluation.

Manual annotation was performed in Fiji using pixel-wise labelling to create binary ground-truth masks distinguishing roots from background agar. Annotation quality was verified through systematic visual overlay inspection of every annotated frame to confirm mask-to-image alignment.

Raw `.png` images and corresponding `.tif` mask files were preprocessed by automatic cropping based on edge detection to remove black borders around the agar plates, followed by padding and tiling into non-overlapping 256×256 pixel patches to accommodate hardware constraints while preserving resolution. Before discarding empty patches, foreground pixels represented less than 2% of the cropped image area, confirming severe class imbalance. Patches lacking foreground across all 15 imaging days were discarded to mitigate background bias and enhance training stability. No data augmentation was applied to maintain a controlled training environment that enables valid model architecture comparison.

The resulting patched dataset dimensions appear below in table 1.

Table 1: Final dataset dimensions after preprocessing and patch extraction.

Dataset split	Number of patches	Number of masks
Training	3,285	3,285
Validation	108	108
Test	161	161

All seven architectures share a common training protocol to enable fair comparison: the Adam optimiser ($\alpha = 10^{-3}$), binary cross-entropy loss, batch size of 8, and five independent runs with matched random seeds for reliable evaluation. F1-score was monitored throughout training. No early stopping was applied, as preliminary experiments confirmed convergence within the per-model training budget without signs of overfitting. The architecture-specific epoch and step counts are summarised in Table 2.

Table 2: Training hyperparameters for all architectures.

Model pair	Epochs	Steps per epoch
U-Net / LSTM-U-Net	3	1,000
ResNet / LSTM-ResNet	4	1,000
ViT, ViT-no-MLP, LSTM-ViT	7	1,500

3.2 U-Net & LSTM-U-Net

The baseline U-Net follows a standard encoder–decoder architecture with four downsampling levels using 3×3 convolutions, batch normalisation, dropout, and max pooling. Filter counts progress from 16 to 256 at the bottleneck, then mirror symmetrically during upsampling. The model processes single RGB images of size $256 \times 256 \times 3$ and outputs binary segmentation masks through a final sigmoid layer, as shown in Figure 2. The architecture contains 1.9M trainable parameters.

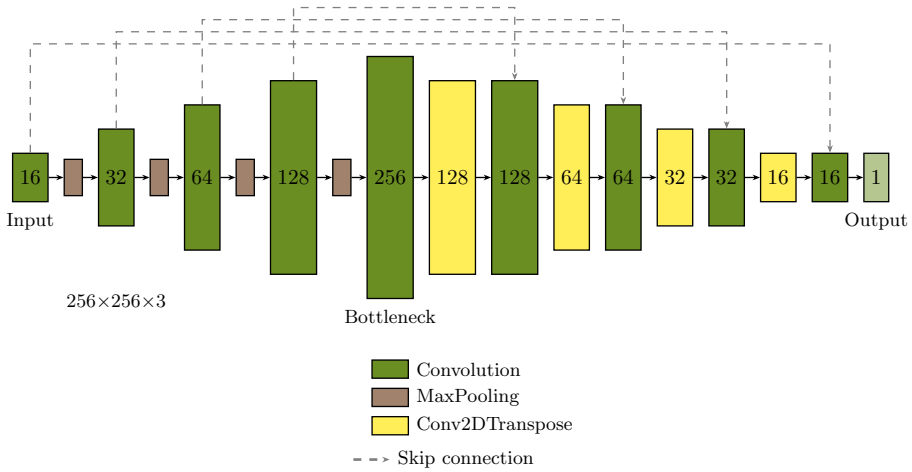


Fig. 2: U-Net architecture

LSTM-U-Net extends the baseline for temporal sequence processing by accepting 15-frame sequences of size $15 \times 256 \times 256 \times 3$. All encoder and decoder convolutions are wrapped in `TimeDistributed` layers, processing each frame independently while preserving temporal ordering. `TimeDistributed` skip connections concatenate temporally-aligned encoder and decoder features at each resolution level, preserving both spatial detail and temporal consistency across frames.

The only structural modification occurs at the bottleneck: the second `Conv2D`(256) is replaced by a `ConvLSTM2D` layer (256 filters, 3×3 kernels, `tanh` activation), followed by batch normalisation wrapped in a `TimeDistributed` layer. The first `Conv2D`(256) is retained, ensuring the bottleneck depth remains identical to the baseline. This minimal change isolates the effect of temporal recurrence from any differences in spatial capacity, allowing a direct controlled comparison of whether temporal memory improves segmentation consistency for roots that disappear and reappear across frames. All other design choices: dropout rates (0.1–0.3 increasing with depth), batch normalisation placement, and ReLU activations, remain unchanged. The final model contains 6M trainable parameters. Figure 3 shows the architecture.

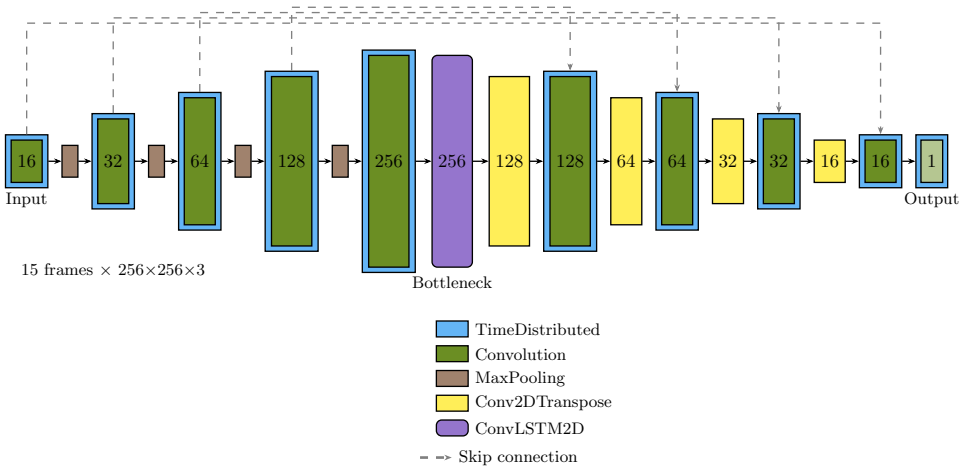


Fig. 3: LSTM U-Net architecture

3.3 ResNet & LSTM-ResNet

Rather than redesigning the ResNet backbone, this study adopted a minimal change strategy in which the static encoder–decoder is reused for each frame and only the second convolution in the bottleneck residual block is replaced by a `ConvLSTM2D` layer to aggregate temporal context. This controlled setup ensures that any observed differences between ResNet and LSTM-ResNet can be attributed solely to temporal recurrence, rather than to additional spatial capacity or architectural complexity.

LSTM-ResNet extends the baseline to short temporal sequences by accepting 15-frame inputs of size $15 \times 256 \times 256 \times 3$. All encoder and decoder convolutions are wrapped in `TimeDistributed` layers, so that each frame is processed through the same shared residual encoder–decoder while preserving temporal ordering. The architecture otherwise remains identical to the static ResNet: an initial 7×7 convolution with stride 2, three residual blocks with progressive downsampling from 64 to 256 filters, and a decoder built from three `Conv2DTranspose` layers that restore spatial resolution while reducing channel depth. In figure 4 the ResNet architecture can be seen.

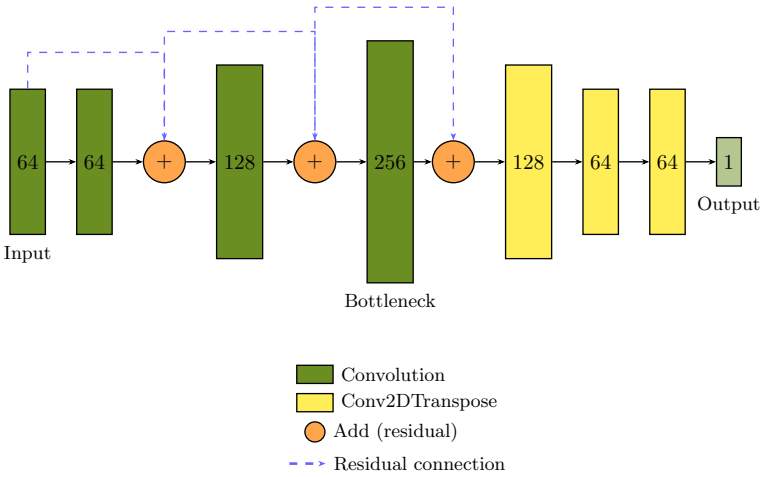


Fig. 4: ResNet architecture

At the bottleneck, within Residual Block 3, the second `Conv2D` (256 filters, 3×3) is replaced by a `ConvLSTM2D` layer (256 filters, 3×3 kernels, `tanh` activation), followed by wrapped batch normalisation (identical to the U-Net approach). Critically, the residual shortcut connection of Block 3 is preserved, meaning the full residual structure of the baseline remains intact. This placement is deliberate, at the bottleneck, spatial resolution is at its lowest (32×32), so the `ConvLSTM2D` can aggregate high-level spatio-temporal features efficiently across all 15 frames before decoding. The decoder then produces per-frame segmentation masks simultaneously via a `TimeDistributed` sigmoid output layer, enabling assessment of temporal consistency across the sequence. The final model contains 4.6M trainable parameters. Figure 5 illustrates the overall architecture.

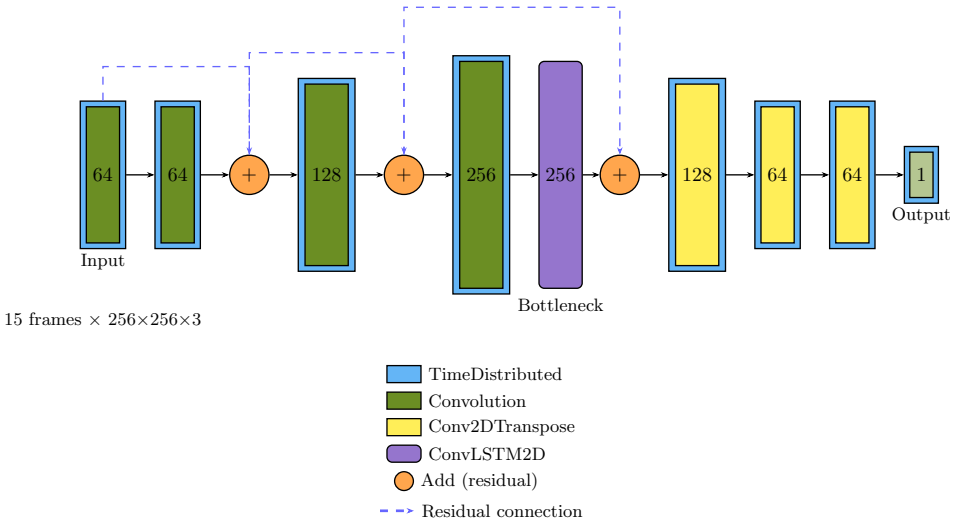


Fig. 5: LSTM-ResNet architecture

3.4 ViT & LSTM-ViT

Unlike the ResNet and U-Net comparisons, which each involve two architectures, the ViT investigation required three: a full ViT with Multi-Layer Perceptron (MLP) block, a ViT without MLP block, and an LSTM-ViT. This was motivated by two distinct questions. The first concerns the MLP block itself: in the standard transformer architecture, each block contains a multi-head self-attention sublayer followed by a two-layer feedforward MLP with `GELU` activation, which projects attention outputs into a higher-dimensional space before projecting back. While this component is well-established for image classification, its necessity for dense prediction tasks such as segmentation is less clear, since spatial feature preservation is prioritised over global semantic abstraction. Following the approach proposed by Hossain et al. [22], removing the MLP block was shown to reduce computational cost without meaningfully degrading segmentation quality. The second question concerns temporal recurrence: as with the ResNet and U-Net pairs, the goal was to test whether a ConvLSTM2D bottleneck improves segmentation consistency for roots that disappear and reappear across frames. However, since LSTM-ViT builds directly on the ViT-no-MLP backbone, the MLP ablation had to be evaluated first. Any performance difference attributed to temporal recurrence must be interpreted knowing what the no-MLP backbone achieves on its own. The three architectures therefore form a structured progression: ViT-MLP establishes the full transformer baseline, ViT-no-MLP isolates the contribution of the MLP block, and LSTM-ViT tests whether temporal recurrence can compensate for or surpass both static variants.

Due to hardware constraints, all three ViT variants process single-channel grayscale images of size $256 \times 256 \times 1$ rather than RGB. This decision was applied consistently

across all three models to ensure a fair comparison, and allowed the model to operate within available memory while maintaining full spatial resolution.

The baseline ViT adapts the standard transformer architecture for image segmentation by replacing traditional patch tokenisation with a convolutional stem. Three successive `Conv2D` layers (64, 128, and 128 filters) downsample the input to a 32×32 feature map, which is then reshaped into a sequence of 1024 tokens with an embedding dimension of 128. Learnable positional embeddings are added to preserve spatial ordering before the sequence is passed through four transformer blocks, each consisting of multi-head self-attention (4 heads) with layer normalisation and a two-layer MLP feedforward network (`GELU` activation, hidden dimension 256). The decoder restores spatial resolution through three `Conv2DTranspose` layers (128→64→32 filters), with a final `Conv2D`(1) sigmoid layer producing binary segmentation masks. The architecture contains **1M** trainable parameters. Figure 6 shows the architecture.

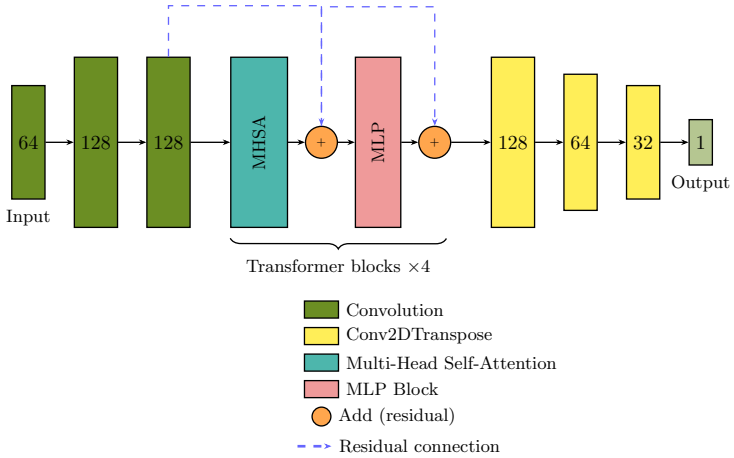


Fig. 6: ViT architecture with MLP block

ViT-no-MLP removes the feedforward MLP block from each transformer block, retaining only the multi-head self-attention sublayer and layer normalisation. All other components remain identical to the baseline: the same convolutional stem, positional embeddings, four transformer blocks, and decoder. This minimal modification reduces computational cost and parameter count while directly testing the contribution of the MLP block to segmentation performance in this domain. The architecture contains **730K** trainable parameters. Figure 7 shows the architecture.

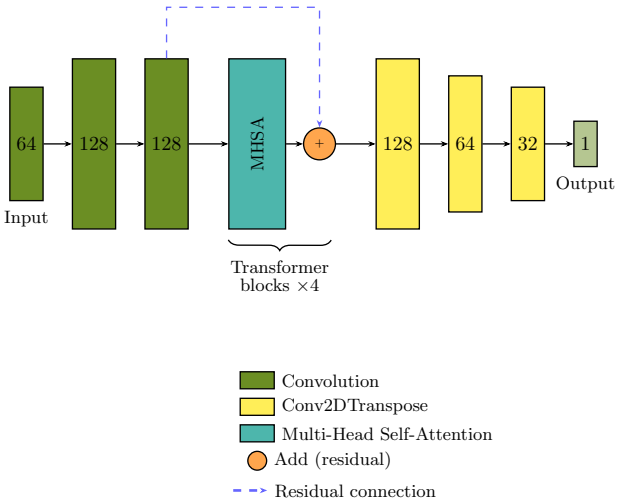


Fig. 7: ViT architecture without MLP block

LSTM-ViT extends the ViT-no-MLP baseline for temporal sequence processing by accepting 15-frame inputs of size $15 \times 256 \times 256 \times 1$. The ViT-no-MLP encoder is wrapped in a `TimeDistributed` layer, so each frame is independently encoded through the same shared convolutional stem, transformer blocks, and spatial reshaping, producing a sequence of feature maps of shape $(15, 32, 32, 128)$. Temporal aggregation is then performed by a `Bidirectional ConvLSTM2D` layer (64 filters per direction, 3×3 kernels, `tanh` activation), which processes the encoded sequence both forward and backward in time, allowing the model to leverage both past and future frame context when producing temporal representations. Since the bidirectional output concatenates both directions, the resulting feature maps have 128 channels, matching the original embedding dimension. A `TimeDistributed Conv2D`(1×1) projection layer consolidates the concatenated features back into the 128-dimensional embedding space before decoding. The decoder is identical to the ViT-no-MLP baseline, wrapped in `TimeDistributed` layers, and produces per-frame segmentation masks simultaneously via a `TimeDistributed` sigmoid output layer. The model contains **1.6M** trainable parameters. Figure 8 shows the architecture.

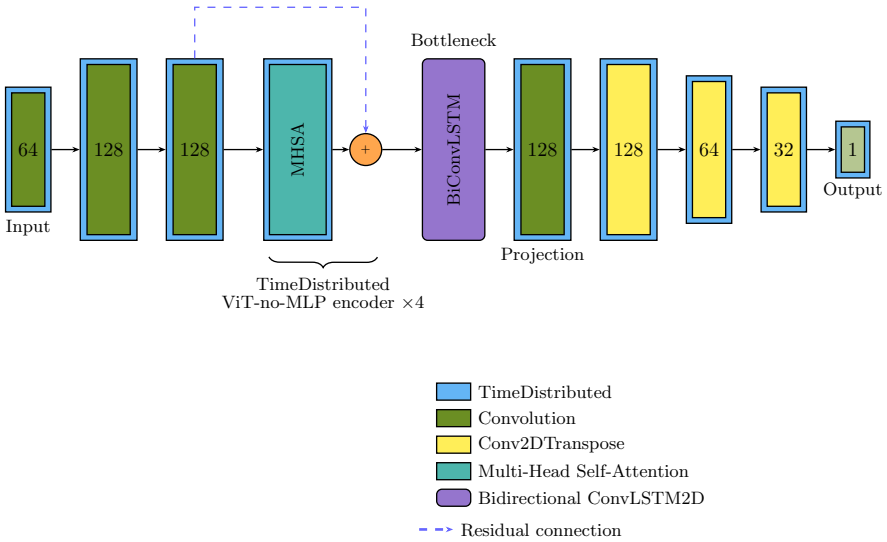


Fig. 8: LSTM-ViT architecture

3.5 Temporal Consistency Score

Standard frame-wise metrics such as F1-score and mIoU evaluate each frame independently and do not capture temporal degradation patterns such as disappearing or disconnected roots between consecutive frames.

To address this, a custom Temporal Consistency Score (TCS) was designed as a cascade-based penalty system that quantifies frame-to-frame prediction coherence against ground truth primary-root growth.

The formula for calculating the Temporal Consistency Score is defined as:

$$\text{TCS} = \frac{1}{N-1} \sum_{t=2}^{N} P(t) \tag{1}$$

where N is the total number of frames and $P(t) \in \{0, 1, 2, 3, 4\}$ is the penalty assigned to frame t using the following cascade priority (evaluated in order, first match applies):

$$P(t) = \begin{cases} 4 & \text{if } \hat{y}_t \leq 0.1 \cdot \max(\mathbf{y}) \\ 3 & \text{if } \dfrac{\hat{y}_t - \hat{y}_{t-1}}{\hat{y}_{t-1}} < -0.03 \ \wedge \ y_t > y_{t-1} \\ 2 & \text{if } \left| \dfrac{\hat{y}_t - \hat{y}_{t-1}}{\hat{y}_{t-1}} \right| \leq 0.03 \ \wedge \ \dfrac{y_t - y_{t-1}}{y_{t-1}} > 0.03 \\ 1 & \text{if } \dfrac{|\hat{y}_t - y_t|}{y_t} > 0.05 \ \wedge \ \hat{y}_t > \hat{y}_{t-1} \\ 0 & \text{otherwise} \end{cases} \tag{2}$$

where $\hat{y}_t$ is the predicted root length at frame t, y_t is the ground truth root length, and $\max(\mathbf{y})$ is the maximum ground truth root length across all frames.

The cascade is evaluated in priority order, so catastrophic disappearance ($P = 4$) is assigned regardless of co-occurring lower-severity conditions. This implicitly weights severe errors above moderate ones in the final mean, providing a built-in severity hierarchy without requiring an additional weighting scheme.

Thresholds were informed by published primary root growth rates of *Arabidopsis thaliana* under standard agar conditions, which range from 3–10% of cumulative root length per day at mid-sequence, with binary mask precision typically within 5% pixel-level tolerance for thin structures. The 5%/3%/10% bounds in equation (2) therefore correspond, respectively, to: predictions outside annotation precision ($P = 1$), regression below biological plausibility ($P = 3$), and stagnation contradicting visible ground-truth growth ($P = 2$).

4 Results

This section reports the performance of all seven architectures evaluated on the held-out test set, comprising a complete 15-day imaging sequence from one independent Agar plate. Each model was trained across five independent runs with matched random seeds in order to critically compare each pair of architectures. Each model was assessed based on the following standard frame-wise metrics: F1-score, mean Intersection over Union (mIoU), precision and recall – as well as the custom Temporal Consistency Score (TCS) defined in Section 3, which checks the length of the primary root in each plant against the ground truth. It is worth noting that no post-processing was done to the predictions, in order to keep the validity of the study.

A consolidated comparison of all seven architectures, averaged across the five independent runs, is presented in Table 3.

Table 3: Summary of averaged metrics across all architectures (mean ± std over 5 runs)

Model	F1-score	mIoU	Precision	Recall	TCS
U-Net	0.60 ± 0.16	0.44 ± 0.15	0.94 ± 0.01	0.45 ± 0.16	2.18 ± 1.15
LSTM-U-Net	0.76 ± 0.01	0.61 ± 0.02	0.91 ± 0.02	0.65 ± 0.03	1.15 ± 0.23
ResNet	0.76 ± 0.02	0.61 ± 0.02	0.91 ± 0.01	0.65 ± 0.03	1.11 ± 0.19
LSTM-ResNet	0.76 ± 0.01	0.62 ± 0.00	0.91 ± 0.01	0.66 ± 0.01	1.50 ± 0.29
ViT	0.69 ± 0.03	0.53 ± 0.04	0.86 ± 0.02	0.58 ± 0.05	2.94 ± 0.15
ViT without MLP	0.71 ± 0.02	0.55 ± 0.02	0.86 ± 0.04	0.60 ± 0.04	2.89 ± 0.21
LSTM-ViT	0.77 ± 0.00	0.62 ± 0.01	0.91 ± 0.01	0.67 ± 0.01	1.11 ± 0.29

4.1 U-Net & LSTM-U-Net

Starting with the comparison between the U-Net and its temporal version, the results showed a clear improvement in overall performance of the model. All metrics can be found in Table 4.

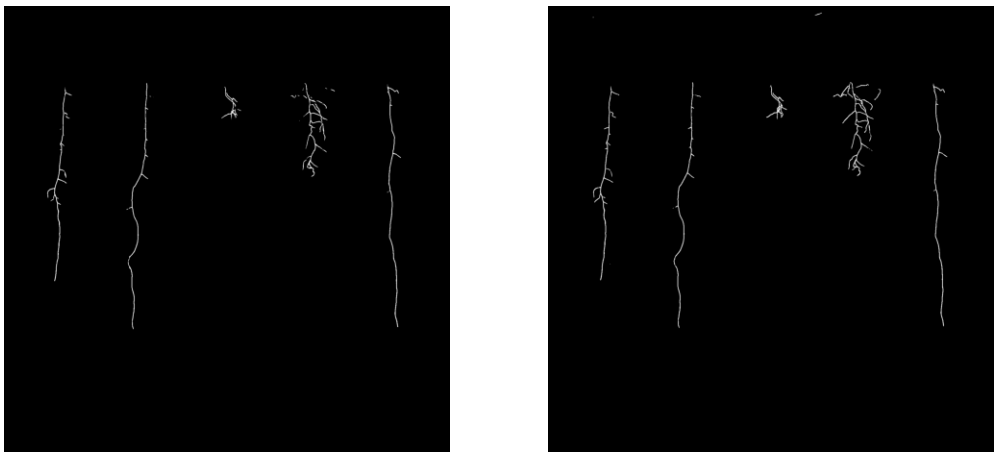
Table 4: U-Net and LSTM-U-Net metrics

Model	Experiment	F1-score	mIoU	Precision	Recall	TCS
U-Net	1	0.64	0.47	0.94	0.48	1.92
	2	0.74	0.59	0.93	0.61	0.91
	3	0.64	0.48	0.93	0.49	1.91
	4	0.66	0.50	0.94	0.51	2.15
	5	0.30	0.17	0.94	0.18	4.00
LSTM-U-Net	1	0.76	0.64	0.91	0.68	1.05
	2	0.76	0.62	0.92	0.65	1.37
	3	0.78	0.61	0.88	0.67	0.77
	4	0.74	0.60	0.92	0.63	1.20
	5	0.75	0.58	0.92	0.61	1.35

On average, the LSTM-U-Net achieved an F1-score of 0.76 compared to 0.60 for the static U-Net, corresponding to an increase of 0.16. Similarly, mIoU improved from 0.44 to 0.61 (+0.17), and recall increased substantially from 0.45 to 0.65 (+0.20), indicating that the temporal model captures a significantly greater proportion of root pixels. Precision decreased marginally from 0.94 to 0.91.

TCS further reinforces this trend, with LSTM-U-Net averaging 1.15 compared to 2.18 for the static model. Notably, the static U-Net failed to converge in Experiment 5 under otherwise identical training conditions, recording an F1-score of 0.30 and a TCS of 4.00. Although the mean TCS improvement was substantial (2.18 to 1.15), this difference did not reach statistical significance (paired t-test, $t = 2.08$, $p = 0.106$), owing to the high variance introduced by the non-convergent Experiment 5 in the static U-Net. The LSTM-U-Net exhibited no such instability across any of its five runs, maintaining an F1-score above 0.74 and a TCS below 1.40 throughout.

Qualitative predictions for a representative sequence are presented in Figure 9, illustrating the improved spatial coherence of the LSTM-U-Net segmentation masks relative to the static baseline from the second experiment.



(a) U-Net prediction (b) LSTM-U-Net prediction

Fig. 9: Qualitative comparison of segmentation predictions - U-Net - LSTM-U-Net

4.2 ResNet & LSTM-ResNet

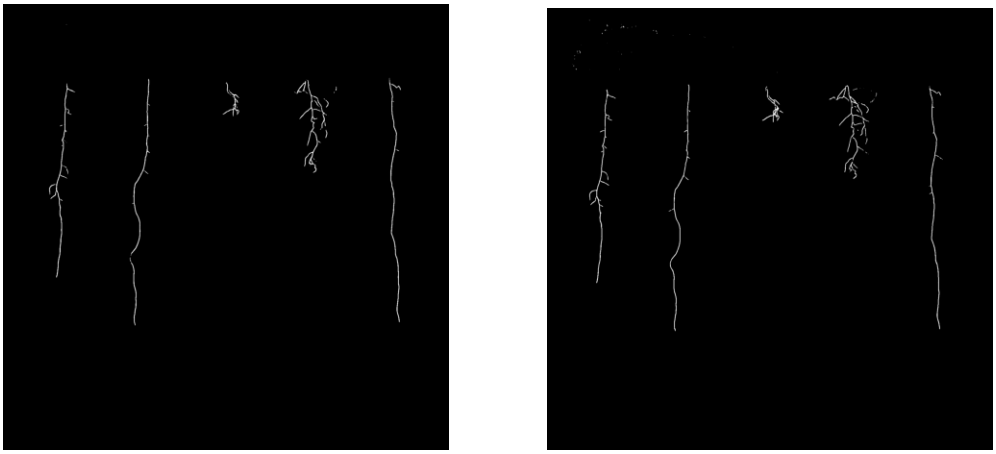
In contrast to the U-Net comparison, the ResNet and LSTM-ResNet architectures yielded near-identical frame-wise performance across all five experiments, as shown in Table 5. Average F1-score increased marginally from 0.756 to 0.764, mIoU from 0.612 to 0.618, and recall from 0.646 to 0.664, with precision remaining effectively unchanged at 0.914 and 0.906 respectively. These differences fall within the natural variance between runs and do not constitute a meaningful improvement.

Table 5: ResNet and LSTM-ResNet metrics

Model	Experiment	F1-score	mIoU	Precision	Recall	TCS
ResNet	1	0.77	0.63	0.90	0.68	1.07
	2	0.77	0.63	0.90	0.67	1.02
	3	0.74	0.59	0.93	0.61	1.22
	4	0.76	0.62	0.92	0.65	0.88
	5	0.74	0.59	0.92	0.62	1.38
LSTM-ResNet	1	0.76	0.62	0.91	0.66	1.72
	2	0.77	0.62	0.91	0.67	1.52
	3	0.76	0.62	0.89	0.67	1.82
	4	0.76	0.61	0.91	0.65	1.04
	5	0.77	0.62	0.91	0.67	1.40

The TCS increased from an average of 1.114 for the static ResNet to 1.500 for LSTM-ResNet, indicating that temporal processing degraded frame-to-frame coherence rather than improving it. This difference was statistically significant (paired t-test, $t = 3.08$, $p = 0.037$).

Qualitative predictions for Experiment 2 are presented in Figure 10, illustrating the comparable segmentation quality between both variants.



(a) ResNet prediction (b) LSTM-ResNet prediction

Fig. 10: Qualitative comparison of segmentation predictions - ResNet - LSTM-ResNet

4.3 ViT & LSTM-ViT

Lastly, the ViT comparison is structured as a three-stage progression (ViT, ViT without MLP, and LSTM-ViT) with all metrics reported in Table 6.

Removing the MLP block produced no meaningful change in performance, with average F1-score increasing marginally from 0.694 to 0.706, mIoU from 0.532 to 0.548, and recall from 0.582 to 0.602, while precision remained effectively unchanged at 0.862 and 0.860 respectively. The TCS showed an equally negligible difference of 2.938 versus 2.894.

The introduction of temporal recurrence in LSTM-ViT, however, produced the most pronounced improvement observed across all three architecture pairs. Compared to ViT without MLP, LSTM-ViT achieved an average F1-score of 0.768 (+0.062), mIoU of 0.624 (+0.076), recall of 0.668 (+0.066), and precision of 0.908 (+0.048). The TCS improved from 2.894 to 1.112 (a reduction of 1.782, representing the largest temporal consistency improvement recorded across all experiments). This difference was highly significant (paired t-test, $t = 10.65$, $p = 0.0004$).

Qualitative predictions for Experiment 2 are presented in Figure 11, illustrating the marked improvement in segmentation quality of the LSTM-ViT relative to both static variants.

Table 6: ViT and LSTM-ViT metrics

Model	Experiment	F1-score	mIoU	Precision	Recall	TCS
ViT	1	0.71	0.55	0.85	0.61	3.11
	2	0.68	0.51	0.87	0.56	2.87
	3	0.65	0.48	0.89	0.51	2.95
	4	0.73	0.58	0.85	0.64	2.72
	5	0.70	0.54	0.85	0.59	3.04
ViT without MLP	1	0.72	0.56	0.83	0.64	2.75
	2	0.69	0.53	0.92	0.55	2.74
	3	0.68	0.52	0.87	0.56	3.15
	4	0.73	0.58	0.84	0.65	2.72
	5	0.71	0.55	0.84	0.61	3.11
LSTM-ViT	1	0.77	0.63	0.92	0.67	0.74
	2	0.77	0.63	0.89	0.68	1.54
	3	0.76	0.61	0.90	0.66	1.18
	4	0.77	0.63	0.91	0.67	1.10
	5	0.77	0.62	0.92	0.66	1.00



(a) ViT no MLP prediction (b) ViT prediction (c) LSTM-ViT prediction

Fig. 11: Qualitative comparison of segmentation predictions - ViT - ViT without MLP - LSTM-ViT

5 Discussion

The results presented in Section 4 reveal that LSTM integration does not uniformly benefit all backbone architectures. Rather, the effectiveness of temporal recurrence depends critically on the underlying architectural design, particularly how information flows through the network during training. This section interprets these findings, examines their architectural and methodological implications, and identifies the conditions under which temporal processing improves segmentation consistency for longitudinal root phenotyping.

5.1 Replacement Strategy for Controlled Architectural Comparison

A central methodological decision in this study was to integrate ConvLSTM2D layers via a *replacement* strategy rather than an *augmentation* strategy. In the replacement approach, the ConvLSTM2D layer substitutes an existing Conv2D layer at the bottleneck, preserving the overall network depth and keeping the architectural change as minimal as possible. The alternative—augmenting the architecture by adding ConvLSTM2D layers on top of existing blocks—would have introduced additional network capacity, making it impossible to attribute performance differences solely to temporal processing.

This distinction is critical for the validity of the comparative analysis. If temporal variants contained additional layers or significantly more parameters than their static baselines, any performance improvement could plausibly be explained by the added representational capacity rather than by temporal recurrence itself. By replacing rather than augmenting, the experimental design isolates a single variable: the ability to model temporal dependencies across frames. For U-Net, this meant replacing the bottleneck Conv2D with ConvLSTM2D while maintaining identical dropout rates, batch normalisation placement, and activation functions. For ResNet, the second Conv2D within Residual Block 3 was replaced with ConvLSTM2D, preserving the residual connection structure. This level of control ensures that the observed differences in TCS and frame-wise metrics can be interpreted as effects of temporal memory rather than confounding architectural changes.

5.2 Residual Connections and ConvLSTM2D Incompatibility

The most counterintuitive finding in this study is that LSTM-ResNet did not improve upon, and in fact degraded, the temporal consistency of the baseline ResNet (Section 3.3). While the frame-wise metrics (F1, mIoU) remained comparable between the two architectures, the TCS scores were consistently worse for LSTM-ResNet, indicating that the ConvLSTM2D layer actively harmed temporal coherence rather than improving it.

The explanation lies in how residual connections interact with ConvLSTM2D during training. In a standard residual block, the output is computed as $\mathcal{F}(x) + x$, where $\mathcal{F}(x)$ is the learned residual function and x is the identity shortcut. When ConvLSTM2D replaces the second convolution within this block, the network retains a direct path from input to output through the shortcut connection. During backpropagation, gradients flow more easily through the identity path than through the ConvLSTM2D branch, which must learn both spatial and temporal representations simultaneously. This creates an optimisation shortcut: the network learns to rely on the identity mapping and does not adequately train the ConvLSTM2D branch.

The consequence at inference is that the ConvLSTM2D operates with undertrained weights. Rather than providing useful temporal context, it introduces noise into the residual function $\mathcal{F}(x)$. The residual addition propagates this noise forward through the decoder, producing frame-to-frame inconsistencies that manifest as degraded TCS. Critically, this effect is masked by the frame-wise metrics: because the identity path

preserves the spatial feature representations learned by the encoder, per-frame segmentation quality remains largely unaffected. Only the temporal consistency metric reveals the problem.

A complementary observation is that the static ResNet baseline already achieved the strongest TCS among static architectures (1.11), matching the LSTM-U-Net and LSTM-ViT temporal variants. This is consistent with the gradient-flow argument above: the residual shortcuts that compromise ConvLSTM2D training also confer the static ResNet with strong frame-wise representational stability, since unobstructed gradient propagation aids convergence on small datasets. The static ResNet therefore approaches the performance ceiling that temporal recurrence is intended to reach, leaving little headroom for ConvLSTM2D to add value, and creating the conditions in which the noise it introduces becomes the dominant effect.

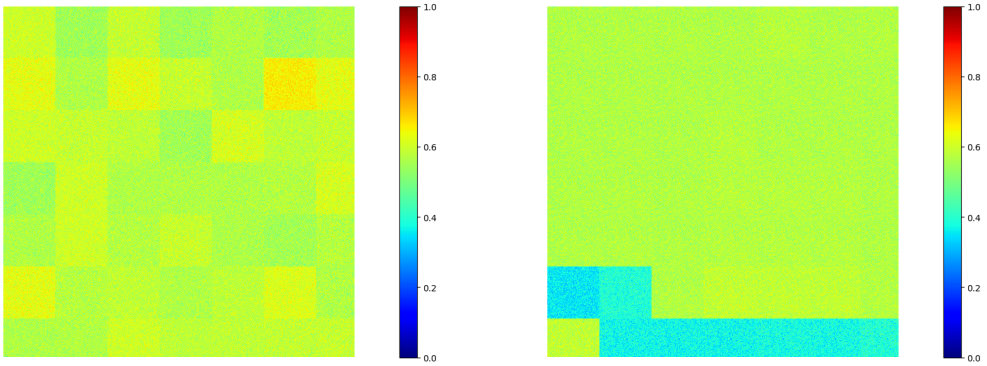
5.3 Explainable AI Analysis

To further understand how temporal recurrence affects each architecture's decision-making RISE saliency maps were generated for all seven models using the day 15 image from the test set. RISE operates by systematically occluding random regions of the input and measuring the resulting change in prediction confidence, producing a per-pixel importance map that indicates which spatial regions most influence the model's segmentation output. Because the input images are patchified into 256×256 tiles before inference, RISE was applied independently to each patch and the resulting saliency maps were reassembled into full-image composites. This patch-wise application imposes a methodological constraint: per-patch normalisation can produce visible tile boundaries in the reassembled composites, so saliency comparisons should be interpreted at the inter-patch (image-level) scale rather than at sub-patch resolution. The resulting visualisations reveal architecture-specific attention patterns that explain the quantitative differences observed in Section 4.

5.3.1 U-Net and LSTM-U-Net

The baseline U-Net produces a saliency map characterised by a checkerboard artefact: each 256×256 tile receives a slightly different overall saliency level, but there is no coherent spatial structure across the image (Figure 12). The model treats every region of the agar plate with roughly equal importance, assigning similar attention to patches containing roots and patches containing only background. This uniform attention pattern is consistent with a static architecture that processes each frame in isolation, without temporal context, the U-Net has no basis for developing a spatial prior about where roots are likely to appear.

The LSTM-U-Net saliency map is qualitatively different. Rather than a checkerboard, it shows a clear spatial gradient: high saliency (warm colours) throughout the upper and middle regions of the plate where root structures are present, dropping sharply to low saliency (cyan/blue) in the bottom portion of the image. Critically, this cold region spans across multiple patch boundaries, indicating that the spatial structure is not an artefact of per-patch normalisation but reflects a genuine, image-level



(a) U-Net RISE mask

(b) LSTM-U-Net RISE mask

Fig. 12: RISE saliency maps for U-Net and LSTM-U-Net

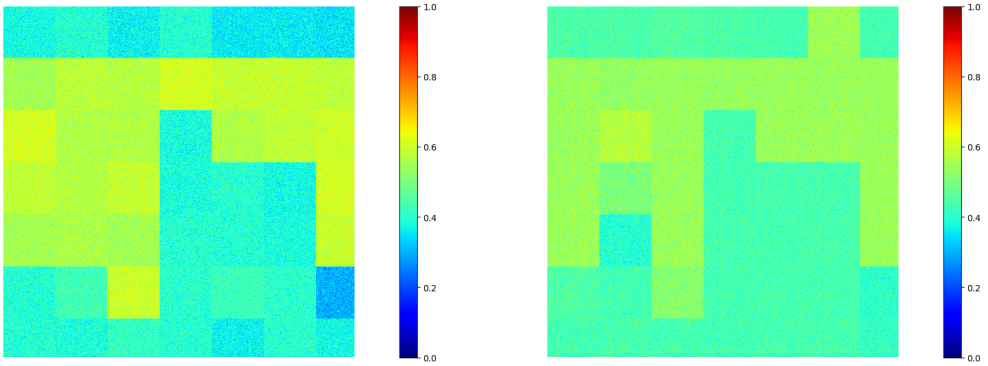
understanding of root distribution. The temporal context provided by the ConvLSTM2D bottleneck has allowed the model to learn that root structures predominantly occupy the upper and central portions of the plate and to suppress attention in regions where roots have not appeared across the 15-frame sequence. This represents a spatial benefit emerging from temporal learning: the ConvLSTM2D does not merely improve when roots are detected across frames, but also refines where the model expects to find them within each frame.

5.3.2 ResNet and LSTM-ResNet

The ResNet saliency map exhibits clear spatial discrimination. Warm patches correspond to the central regions of the image where root structures are concentrated, while cooler patches at the top edge and bottom corners indicate reduced attention in areas with less root content (Figure 13). The residual architecture's skip connections enable effective gradient flow during training, and the resulting model has learned to focus its attention on relevant regions while suppressing irrelevant ones.

The LSTM-ResNet saliency map, by contrast, shows a flattened contrast profile. The difference in saliency between root-containing and empty patches is reduced compared to the baseline, with the model assigning more uniform importance across the image, including to patches with no root structures. Rather than developing a refined spatial prior as LSTM-U-Net did, the LSTM-ResNet's attention has become less discriminating than the baseline.

This observation provides direct visual evidence for the mechanism described in Section 5.2. The residual shortcut allows the network to bypass the ConvLSTM2D during training, leaving it with undertrained weights that introduce noise rather than useful temporal signal at inference. The RISE maps make this effect visible: the ConvLSTM2D washes out the spatial discrimination that the residual architecture had already developed, producing a more diffuse attention pattern that does not distinguish between root-containing and empty regions. Where LSTM-U-Net gained spatial



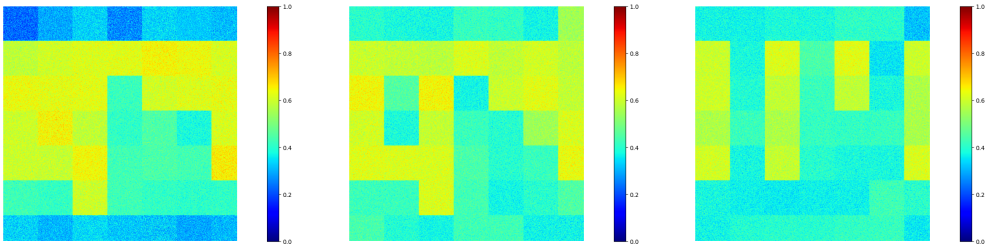
(a) ResNet RISE mask (b) LSTM-ResNet RISE mask

Fig. 13: RISE saliency maps for ResNet and LSTM-ResNet

structure that U-Net lacked, LSTM-ResNet lost spatial structure that ResNet already had—a finding consistent with the degraded TCS observed in the quantitative results.

5.3.3 ViT-MLP, ViT-no-MLP, and LSTM-ViT

The three ViT variants produce the most informative progression of saliency patterns (Figure 14). ViT-MLP shows strong spatial discrimination with a horizontal banding pattern: the top rows of the image receive low saliency, a broad middle band where roots are concentrated shows warm (yellow/orange) values, and the bottom drops back to cool colours. The MLP's nonlinear feature transformation enables focused, spatially discriminating attention that clearly separates root-containing from empty regions. Among the static baselines, ViT-MLP produces the highest-contrast saliency map.



(a) ViT RISE mask (b) ViT no MLP RISE mask (c) LSTM-ViT RISE mask

Fig. 14: RISE saliency maps for baseline ViT, ViT without MLP block and LSTM-ViT

Removing the MLP block disrupts this spatial focus. The ViT-no-MLP saliency map loses the coherent horizontal gradient, with warm and cool patches distributed

more irregularly across the image, resembling the scattered pattern observed in the static U-Net. The model's capacity for per-token feature refinement has been reduced, degrading its ability to discriminate between root-containing and empty regions. However, a new pattern begins to emerge: vertical columns of elevated saliency appear between the plant positions. Although the overall attention is less focused, the model has started to capture the columnar structure of root growth, where roots extend predominantly downward from the seed.

LSTM-ViT combines the strengths of both preceding variants. The Bidirectional ConvLSTM2D restores the focused, high-contrast attention that the MLP had provided while simultaneously preserving and enhancing the vertical stripe pattern that emerged in ViT-no-MLP. The resulting saliency map shows vertical corridors of high importance that align with root growth trajectories between plant positions, with suppressed attention at the top edge and bottom corners where no root structures are present. This dual benefit is consistent with LSTM-ViT achieving the best TCS and the strongest frame-wise metrics across all architecture pairs. The ConvLSTM2D does not merely replace the MLP's spatial capacity; it augments it with temporally informed directional attention that the MLP, operating on single frames, could never develop.

5.4 Vision Transformer as the Strongest Beneficiary of Temporal Integration

Among the three architecture pairs evaluated, LSTM-ViT produced the largest improvements in both frame-wise metrics and temporal consistency, achieving the lowest average TCS of **1.112** across all experiments. This stands in contrast to U-Net, where temporal integration yielded moderate but consistent improvements, and ResNet, where it was counterproductive.

Several architectural properties of the Vision Transformer may explain why it benefits most from temporal recurrence. First, the ViT processes input as a sequence of patch tokens through self-attention, a mechanism that captures global spatial relationships but has no inherent notion of temporal ordering across frames. Unlike convolutional architectures, where local receptive fields and pooling operations impose an inductive bias toward spatial locality, the ViT's attention mechanism distributes information globally within each frame but treats consecutive frames as entirely independent inputs. The addition of ConvLSTM2D at the bottleneck provides precisely the missing temporal inductive bias, allowing the model to carry information about root structures from earlier frames into later ones.

Second, the ViT variant used in this study employs a Bidirectional ConvLSTM2D, which processes the 15-frame sequence in both forward and backward directions. The motivation for this choice mirrors the strategy a human annotator would naturally adopt when segmenting roots from agar plate imagery. When an annotator encounters an ambiguous root at a given timepoint, for instance, a faint lateral root, the natural instinct is to examine neighbouring frames (one day before and one day after) to determine whether a root structure is genuinely present. Bidirectional processing replicates this reasoning computationally: the forward pass carries information about root structures observed in earlier frames, while the backward pass provides context

from later frames where the same root may be more clearly visible. This allows the model to resolve ambiguities at any given timepoint using temporal context from both directions, producing more stable segmentation across the entire sequence.

5.5 Limitations

Several limitations should be considered when interpreting these results. First, all three LSTM-enhanced architectures converged to similar F1-scores ($\sim$0.77), suggesting a possible upper bound imposed by the dataset rather than by the models: a 105-image annotated subset, while temporally complete, may be insufficient to differentiate the architectures further on frame-wise metrics. Second, this study compared architectural temporal integration against frame-independent baselines but did not directly benchmark against post-processing approaches such as morphological smoothing or Kalman-filter-based prediction refinement; integrating both strategies remains an open question. Third, although the parameter counts of LSTM variants approximately tripled relative to baselines, the lab GPU (NVIDIA RTX 6000 Ada, 48 GB VRAM) accommodated training and inference comfortably, so computational cost was not a limiting factor in this work, though it would be relevant for deployment on resource-constrained phenotyping platforms. Finally, RISE saliency analysis was performed on a single representative day; full longitudinal saliency evolution across the 15-day sequence is left for future investigation.

6 Conclusion

This study investigated whether integrating ConvLSTM2D layers into U-Net, ResNet, and Vision Transformer architectures can improve temporally consistent segmentation of *Arabidopsis thaliana* roots beyond what static frame-wise architectures provide. Three contributions emerge. First, LSTM integration is not a universal enhancement: it improved both segmentation quality and temporal coherence in U-Net (TCS 2.18 $\rightarrow$ 1.15) and ViT (TCS 2.89 $\rightarrow$ 1.11, the largest improvement observed), but degraded temporal consistency in ResNet (TCS 1.11 $\rightarrow$ 1.50), where residual shortcut connections prevented the ConvLSTM2D branch from training effectively. Second, we proposed the Temporal Consistency Score (TCS), a cascade-based penalty metric grounded in *Arabidopsis* daily growth rates, providing a biologically motivated complement to frame-wise IoU/Dice/F1 for evaluating sequence-level segmentation stability. Third, RISE-based explainability analysis revealed mechanistic differences in how each architecture's attention is reshaped by temporal context, supporting the broader claim that architectural compatibility with recurrent layers should be assessed alongside performance metrics. For phenotyping labs deploying temporal segmentation pipelines, the practical implication is that architectures with unobstructed bottleneck information flow, particularly Vision Transformers, are the most suitable hosts for recurrent temporal layers, while residual architectures may require modified integration strategies to benefit from temporal recurrence.

6.1 Further Research

Several directions remain for further investigation, listed in approximate order of expected impact. First, modified residual integration strategies, gating the residual shortcut or placing temporal layers outside the residual block, would directly address the ResNet incompatibility identified here and is the most theoretically grounded extension. Second, cross-species and cross-modality validation would establish whether the architecture-dependent benefits generalise beyond *Arabidopsis thaliana* agar plate imagery, with the highest practical value for the phenotyping community. Third, alternative temporal mechanisms such as attention-based temporal fusion or temporal convolutional networks could be compared against ConvLSTM2D to clarify whether the observed benefits are LSTM-specific or generalise across recurrent designs. Finally, sequence-length scaling studies beyond 15 frames would inform pipeline design for longer-duration growth experiments.

Availability of data and materials

The code for all architectures, training pipelines, and the Temporal Consistency Score implementation is available at https://github.com/TudorPitulice234803/cnn_lstm_root_architecture. The dataset used in this study was acquired at the National Plant Eco-phenotyping Center (NPEC) and is available from the corresponding author upon reasonable request.

References

[1] Lombardi M, Gennaro S, Iannacone R. Determinants of root system architecture for future-ready, stress-resilient crops. Physiologia Plantarum. 2021;172(3):2070–2084. https://doi.org/10.1111/ppl.13439.

[2] Justamante MS, colleagues. Target genes modulating root architecture in abiotic stress. Scientific Reports. 2025;15:Article 97266. https://doi.org/10.1038/s41598-025-97266-y.

[3] Gautam K, colleagues. Root architecture and carbon sequestration of fast-growing trees in agroforestry systems. Frontiers in Agronomy. 2025;5:1597122. https://doi.org/10.3389/fagro.2025.1597122.

[4] Gaggion N, others. ChronoRoot: High-throughput phenotyping by deep segmentation networks reveals novel temporal parameters of plant root system architecture. GigaScience. 2021;10(7):giab052. https://doi.org/10.1093/gigascience/giab052.

[5] Lobet G, Draye X. The state of the art in root system architecture image analysis using artificial intelligence: A review. Plant Phenomics. 2025;https://doi.org/10.34133/plantphenomics.0178.

[6] Seidenthal K, et al. Iterative image segmentation of plant roots for high-throughput trait extraction. Scientific Reports. 2022;12:16685. https://doi.org/10.1038/s41598-022-19754-9.

[7] Petsiuk V, Das A, Saenko K. RISE: Randomized Input Sampling for Explanation of Black-box Models. In: Proceedings of the British Machine Vision Conference (BMVC); 2018. p. 151.

[8] Yasrab R, Pound MP, Wilson MC, French AP, Pridmore TP. RootNav 2.0: Deep learning for automatic navigation of complex root architectures. GigaScience. 2019;8(11):giz123. https://doi.org/10.1093/gigascience/giz123.

[9] Shen C, Liu L, Zhu L, Kang J, Wang N, Shao L. High-Throughput in situ Root Image Segmentation Based on the Improved DeepLabv3+ Method. Frontiers in Plant Science. 2020;11:576791. https://doi.org/10.3389/fpls.2020.576791.

[10] Li Y, colleagues. An improved U-Net-based in situ root system phenotype segmentation model. Frontiers in Plant Science. 2023;14:1115713. https://doi.org/10.3389/fpls.2023.1115713.

[11] Baykalov P, Bussmann B, Nair R, Smith AG, Bodner G, Hadar O, et al. Semantic segmentation of plant roots from RGB (mini-)rhizotron images—generalisation potential and false positives of established methods and advanced deep-learning models. Plant Methods. 2023;19(1):122. https://doi.org/10.1186/s13007-023-01101-2.

[12] Huang Y, Sun Y, Liu J, Zhao S, Liu X, Wu J. Ginger seeding detection and shoot orientation discrimination using an improved YOLOv8 model. Frontiers in Plant Science. 2023;14:1147034. https://doi.org/10.3389/fpls.2023.1147034.

[13] Yu Q, Tang H, Zhu L, Zhang W, Liu L, Wang N. A method of cotton root segmentation based on edge devices. Frontiers in Plant Science. 2023;14:1122833. https://doi.org/10.3389/fpls.2023.1122833.

[14] Smith AG, Han E, Petersen J, Olsen NAF, Giese C, Athmann M, et al. RootPainter: deep learning segmentation of biological images with corrective annotation. New Phytologist. 2022;236(2):774–791. https://doi.org/10.1111/nph.18387.

[15] Cai X, Lu Y, Wu Y, Yang H, Tian W. CUDU-Net: Collaborative upsampling decoder U-Net for leaf vein segmentation. Digital Signal Processing. 2024;144:104287. https://doi.org/10.1016/j.dsp.2023.104287.

[16] Wang L, Whalley WR, Miller AJ, Mooney SJ, Sturrock CJ. Improving the segmentation of fine roots in X-ray computed tomography images through deep learning. Sensors. 2025;25(16):4956. https://doi.org/10.3390/s25164956.

[17] Hu Y, Liu G, Chen Z, Liu J, Guo J. Lightweight one-stage maize leaf disease detection model with knowledge distillation. Frontiers in Plant Science. 2023;14:1231903. https://doi.org/10.3389/fpls.2023.1231903.

[18] Yu S, Xie L, Huang Q. Inception convolutional vision transformers for plant disease identification. Internet of Things. 2023;21:100650. https://doi.org/10.1016/j.iot.2022.100650.

[19] Yasrab R, colleagues. Bridging phenotype–genotype gap: A CNN–LSTM based plant root phenotyping system. bioRxiv. 2020;p. 2020.05.03.075184. https://doi.org/10.1101/2020.05.03.075184.

[20] Tian X, et al. U + LSTM-F: A data-driven growth process model of rice seedlings using time-series phenotyping. Ecological Informatics. 2024;79:102360. https://doi.org/10.1016/j.ecoinf.2023.102360.

[21] Zhou L, colleagues. Phenotypic analysis of diseased plant leaves using time-series deep learning. Computers and Electronics in Agriculture. 2023;209:107821. https://doi.org/10.1016/j.compag.2023.107821.

[22] Hossain MR, Islam MR, et al. A novel hybrid ViT-LSTM model with explainable AI for brain stroke detection and classification in CT images: A case study of Rajshahi region. Computers in Biology and Medicine. 2025;6(1):1–15. https://doi.org/10.1016/j.compbiomed.2025.109711.

[23] Chandra AL, Desai SV, Guo W, Balasubramanian VN. Computer vision with deep learning for plant phenotyping in agriculture: A survey. arXiv preprint. 2020;arXiv:2006.11391. [cs.CV].